Azure Databricks – Hands-on Assessment

Scenario: Smart Parking Management System

Participants will work with parking transactions, perform SQL and PySpark operations, manage Delta tables, implement MERGE, explore Time Travel, create views, and build a basic streaming ETL pipeline.

No CSV, JSON, or external files are required. All assessment data is created directly inside Azure Databricks using SQL.

Part 1 – Dataset preparation

INSTRUCTOR SETUP – RUN BEFORE THE ASSESSMENT

Step 1 – Create the source Delta table

```
%sql
CREATE OR REPLACE TABLE parking_sessions_source (
    session_id INT,
    zone_id STRING,
    vehicle_type STRING,
    duration_hours INT,
    parking_fee INT,
    status STRING
)
USING DELTA;

INSERT INTO parking_sessions_source VALUES
(101, 'Z01', 'Car', 3, 150, 'Completed'),
(102, 'Z02', 'Bike', 2, 40, 'Completed'),
(103, 'Z03', 'Car', 5, 250, 'Completed'),
(104, 'Z01', 'SUV', 4, 240, 'Active'),
(105, 'Z04', 'Bike', 1, 20, 'Completed'),
(106, 'Z02', 'Car', 6, 300, 'Completed'),
(107, 'Z03', 'SUV', 2, 120, 'Cancelled'),
(108, 'Z05', 'Car', 3, 150, 'Completed'),
(109, 'Z01', 'Bike', 4, 80, 'Completed'),
(110, 'Z04', 'SUV', 5, 300, 'Active'),
(111, 'Z02', 'Car', 2, 100, 'Completed'),
```

```
(112, 'Z05', 'Bike', 3, 60, 'Cancelled'),
(113, 'Z03', 'Car', 7, 350, 'Completed'),
(114, 'Z01', 'SUV', 3, 180, 'Completed'),
(115, 'Z04', 'Bike', 2, 40, 'Active'),
(116, 'Z05', 'Car', 4, 200, 'Completed'),
(117, 'Z02', 'SUV', 6, 360, 'Completed'),
(118, 'Z03', 'Bike', 1, 20, 'Completed'),
(119, 'Z04', 'Car', 5, 250, 'Completed'),
(120, 'Z05', 'SUV', 2, 120, 'Active');
```

Step 2 — Create a working Delta table

Participants will perform updates, deletes, and MERGE operations on this copy.

```
%sql
CREATE OR REPLACE TABLE parking_sessions_work
USING DELTA
AS
SELECT * FROM parking_sessions_source;
```

Keep `parking_sessions_source` append-only for the streaming exercises.

Step 3 — Create the zone master table

```
%sql
CREATE OR REPLACE TABLE parking_zones (
    zone_id STRING,
    zone_name STRING,
    city STRING,
    capacity INT
)
USING DELTA;

INSERT INTO parking_zones VALUES
('Z01', 'Central Plaza', 'Hyderabad', 100),
('Z02', 'Metro Parking', 'Bengaluru', 150),
('Z03', 'City Mall', 'Mumbai', 200),
('Z04', 'Airport Parking', 'Delhi', 250),
('Z05', 'Tech Park', 'Pune', 120);
```

Step 4 — Prepare incoming records for MERGE

```
%sql
CREATE OR REPLACE TEMP VIEW incoming_parking_sessions AS
SELECT *
FROM VALUES
(102, 'Z02', 'Bike', 3, 60, 'Completed'),
(104, 'Z01', 'SUV', 5, 300, 'Completed'),
(121, 'Z03', 'Car', 4, 200, 'Completed'),
(122, 'Z05', 'Bike', 2, 40, 'Active')
AS t(
    session_id,
    zone_id,
    vehicle_type,
    duration_hours,
    parking_fee,
    status
);
```

Part 2 — Assessment questions

Each question carries 5 marks. Participants must execute the code and demonstrate the results. No solutions are provided.

Section A — SQL and DataFrames (20 marks)

1. Display all records from `parking_sessions_source`. Retrieve only `session_id`, `vehicle_type`, and `parking_fee`.
2. Find all completed parking sessions where the fee is greater than ₹100.
3. Calculate the total parking fees and number of completed sessions for each vehicle type using `GROUP BY`.
4. Using PySpark, read `parking_sessions_source` into a DataFrame, add a column named `fee_category` (`High` for fees of ₹200 or more, otherwise `Low`), and display the result.

Section B — Delta Tables and CRUD (20 marks)

5. Create a new managed Delta table named `parking_summary` using CTAS, containing only completed sessions.

6. Insert a new record into `parking_sessions_work` with session ID 123, zone Z01, vehicle type Car, duration 3 hours, fee ₹150, and status Active.
7. Update session 104 to `Completed` and increase its parking fee to ₹280.
8. Delete all Cancelled sessions from `parking_sessions_work`. Display the remaining records.

Section C — MERGE and Delta Operations (20 marks)

9. Display `incoming_parking_sessions` and identify which records already exist in `parking_sessions_work`.
10. Perform a `MERGE` operation using `session_id` to update existing records and insert new records.
11. Use `DESCRIBE HISTORY` to display the transaction history of `parking_sessions_work`.
12. Use Time Travel to retrieve the table version immediately before the MERGE operation. Compare session 102 before and after MERGE.

Section D — Tables, Views and Optimization (20 marks)

13. Create a permanent view named `completed_parking_view` containing only completed sessions from `parking_sessions_work`.
14. Create a temporary view containing parking sessions from zone Z01 and a global temporary view containing sessions with parking fees above ₹200. Query both views.
15. Join `parking_sessions_work` with `parking_zones` and display the zone name, city, vehicle type, and parking fee.
16. Run `DESCRIBE DETAIL` on `parking_sessions_work`, execute `OPTIMIZE`, and compare the file count before and after optimization. Explain why a small dataset might show no significant change.

Section E — Structured Streaming and DLT (20 marks)

17. Using PySpark's built-in `rate` source, generate five streaming records per second and display the stream.
18. Apply a streaming transformation that creates an `amount` column from `value` and filters records with an amount greater than ₹300.
19. Create a Lakeflow/DLT SQL pipeline with a Bronze streaming table reading from `parking_sessions_source` and a Silver streaming table retaining only completed sessions.
20. Create a Gold materialized view that calculates total completed sessions and total parking revenue by zone. Run the pipeline and verify the Gold output.